

A Workflow and Structure for the Software

This project combines a customised desktop medical imaging viewer with a web platform designed to manage and access medical scan files. The goal is to build something that works as a connected workflow rather than a collection of separate tools. The current focus is on validating the workflow and integration between the desktop viewer and the web platform before considering production deployment.

For now, the focus is on getting a clean, working prototype running locally that can be tested end to end and demonstrated internally.

Desktop Viewer

The viewer is based on MRICroGL version 1.2, an open-source medical imaging tool written in Pascal. Rather than building a viewer from scratch, the existing MRICroGL codebase is being customised and adapted to support the intended workflow and branding requirements.

The changes being made include:

- Adding MedMarvel branding to the interface, including the company name and product identity
- Maintaining all source code in a private GitHub repository under the MedMarvel organisation
- Changing the defaults in the viewer as per the company's requirements

Web Platform

The web platform is a separate application that sits alongside the desktop viewer. Its purpose is straightforward: give scientists a place to manage patient scan files, and give doctors and patients a way to access the results.

There are three types of users:

- Scientists: log in, upload scan files, process them in the desktop viewer, and save results back to patient records
- Doctors and Patients: log in and view the processed scans associated with their records (read-only access)
- Admin: manage users and oversee patient records

How It Works (from the view of the different users)

Scientist

- Logs into the web platform
- Opens a patient record and uploads the relevant scan file
- Clicks “Open in Viewer”, which launches the MRICroGL desktop app with the scan already loaded (this will require a custom Windows extension to be developed at a later stage)
- Processes and overlays the scan as needed
- Saves the processed result back to the patient record on the platform

Doctor / Patient

- Logs into the web platform
- Views their profile and the scan results associated with their records
- Cannot upload or modify any files

File Storage and Organisation

All scan files, both input and output, are stored on the cloud server and organised by patient.

Input files (raw scans)

- All raw scan files live on the cloud server
- Each scan is tagged to a specific patient
- When a scientist opens a patient record, they see only that patient's scans
- They select a scan and open it in the viewer

Output files (processed results)

- After the scientist finishes in the viewer, they save the processed result
- That output file gets saved back under the same patient's record
- Each patient therefore has their own folder containing both their raw input scans and their processed output files

The intended structure is as follows:

Database and File Management

The PostgreSQL database does not directly store the medical scan files themselves. Instead, it stores metadata and references associated with each file.

The actual scan files (such as .nii.gz files and processed outputs) are stored separately on local storage during development and later on cloud storage (AWS S3 planned for production).

Each patient record in the database is linked to:

- Raw input scan files
- Processed output files
- Upload and processing timestamps
- The user who uploaded or processed the file
- File locations/paths on the storage server

This allows the system to organise scans by patient while maintaining a clear relationship between users, patient records, raw scans, and processed results.

Security Considerations

The current phase focuses on local prototype development and workflow validation.

A production deployment would require additional security measures including:

- secure authentication
- encrypted file access
- access logging
- compliance considerations for handling sensitive medical data

Tech Stack

- React: building the web platform frontend
- Python with FastAPI: handling backend logic, API endpoints, and database communication
- PostgreSQL: storing user accounts, patient records, and file metadata
- Local file storage during development, with AWS S3 intended for production
- JWT authentication: managing user login and role-based access
- Custom URL protocol (mrirogl://): launching the desktop viewer directly from the website (can be done using the windows extension)

Note on Scope

The goal for this phase is to have everything working locally on a single development machine.

This includes:

- the frontend on react
- backend API
- database
- desktop viewer
- local file storage

This is to validate workflow integration between the web platform and the desktop viewer before cloud deployment and production hardening. Cloud infrastructure during deployment would need to be procured separately, and additional security hardening would need to be developed and tested before any public or production deployment.

Once the prototype is tested and approved internally, later stages could involve:

- cloud deployment and improved security measures. |

Deployment decisions and infrastructure planning would be handled separately during later stages of development.

